

Technical Leadership Study Workbook

A 10-week project-based curriculum in DevOps, architecture, observability, reliability, SRE, and technical decision-making

PROGRAM INTENT

Build repeatable technical-leadership skills by designing, implementing, observing, testing, and explaining one small production-style .NET system. This is a portfolio-oriented learning program, not certification preparation.

TIME COMMITMENT

Plan for approximately 5-7 hours per week. Week 8 and Week 10 may take closer to 7-8 hours because they integrate several earlier skills.

HOW TO USE THIS WORKBOOK

Complete the work in sequence. Do the quiz before reading the answer key. Prefer written artifacts over passive reading. Keep all deliverables in one Git repository so the final result becomes a compact technical architecture portfolio.

Program Overview

The program uses a single small .NET service as a learning vehicle. Each week adds a new engineering capability and a new communication artifact. By the end, you will have demonstrated that you can move from code-level implementation to system architecture, delivery design, reliability reasoning, and executive-level decision framing.

Week	Focus	Primary Deliverable	Target Time
1	Think in Delivery Systems	One-page Software Delivery System Map with three sections: Control, Artifact, and Verification	5-6 hours
2	Continuous Integration and Reproducible Builds	ADR-001: Build Artifacts Are Immutable	6-7 hours
3	Continuous Deployment and Artifact Promotion	A diagram titled "Deployment Success vs	6-7 hours
4	Architecture Communication and ADRs	A mini architecture packet: 3 diagrams plus ADR-002 and ADR-003	5-6 hours
5	Observability with OpenTelemetry	An Observability Contract defining required deployment attributes, request correlation, critical workflow metrics, and error context	6-7 hours
6	Reliability Engineering and Failure Modes	Failure Mode and Effects Analysis plus a one-page "Reliability Improvements" backlog ranked by risk	5-6 hours
7	Site Reliability Engineering: SLI, SLO, Error Budgets, and Toil	One-page Reliability Scorecard plus a ranked Toil Reduction Backlog	5-6 hours

8	Environment Certification Service	Working certification prototype plus ADR-004: Validation Is Independent of Deployment Execution	7-8 hours
9	Architecture Proposal and Decision Enablement	Two-page proposal, one-page executive version, and 60-second decision pitch	5-6 hours
10	Technical Leadership Simulation	Capstone architecture review deck/packet and completed final assessment described below	6-8 hours

Program Rules

- [] Build one coherent repository rather than ten disconnected exercises.
- [] Do not count reading as completion unless the week's artifact also exists.
- [] Write quiz answers from memory before checking the answer key.
- [] Prefer explicit tradeoffs over "best practice" claims without context.
- [] Keep technical work generic enough to demonstrate transferable skill, not only knowledge of one company system.
- [] At the end of each week, commit the deliverable and write a short retrospective.

Portfolio Repository Structure

Suggested structure:

```

/src
/tests
/pipelines
/docs/diagrams
/docs/adrs
/docs/reliability
/docs/proposals
/docs/week-01 ... /docs/week-10
README.md

```

WEEK 1

Think in Delivery Systems

Move from thinking about individual pipelines to reasoning about software delivery as an end-to-end system.

Suggested time	Primary output	Status
5-6 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Explain the difference between build, deployment, validation, and release acceptance.
- ☐ Map the path from source change to running software.
- ☐ Identify where state changes, human intervention, and failure detection occur.
- ☐ Distinguish prevention controls from detection controls.

Resources

Explore Azure Pipelines (17 min) - Short orientation to Azure Pipelines terminology and core concepts.
<https://learn.microsoft.com/en-us/training/modules/explore-azure-pipelines/>

DevOps foundations (Selected modules) - Review DevOps as a system of people, process, automation, feedback, and continuous delivery.
<https://learn.microsoft.com/en-us/training/paths/devops-foundations-core-principles-practices/>

Azure DevOps overview (Reference) - Use as a reference for CI/CD and delivery-system concepts.
<https://learn.microsoft.com/en-us/devops/overview/>

Hands-on lab

- ☐ 1. Create a small .NET Web API with GET /health, GET /policies/{id}, and POST /policies.
- ☐ 2. Create a Git repository, README, unit-test project, build script, and version number.
- ☐ 3. Add one configuration value that differs between DEV and QA.
- ☐ 4. Do not automate deployment yet. Manually draw the lifecycle from developer commit through release acceptance.
- ☐ 5. Mark each place where state changes, human intervention occurs, failures can be introduced, and failures can be detected.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which parts of the delivery process are deterministic today?

Where does your design rely on a person remembering to do something?

Which failure could travel the farthest before being detected?

Knowledge check

1. What is the difference between continuous integration and continuous delivery?

2. Why is a deployment pipeline more than a build script?

3. Why can human intervention increase deployment risk?

4. What is the difference between preventing a failure and detecting one?

5. What information would you need to identify exactly what code is running in QA?

6. Why can deployment validation be treated separately from deployment execution?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 2

Continuous Integration and Reproducible Builds

Create a build process that reliably turns source into a tested, traceable artifact.

Suggested time	Primary output	Status
6-7 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Build a pipeline-as-code CI workflow.
- ☐ Explain immutable artifacts and provenance.
- ☐ Understand self-hosted agent dependencies and risks.
- ☐ Diagnose build, test, and package failures at the correct stage.

Resources

AZ-400: Implement CI with Azure Pipelines and GitHub Actions (Use selected modules) - Primary learning path for pipeline configuration and agent management.

<https://learn.microsoft.com/en-us/training/paths/az-400-implement-ci-azure-pipelines-github-actions/>

Azure Pipelines agents (Reference) - Pay particular attention to hosted vs. self-hosted agent responsibilities.

<https://learn.microsoft.com/en-us/azure/devops/pipelines/agents/agents>

Hands-on lab

- ☐ 1. Create a pipeline with Restore -> Compile -> Unit Tests -> Package -> Publish Artifact.
- ☐ 2. Ensure failed tests prevent artifact publication.
- ☐ 3. Stamp the artifact with a version that can be traced back to the commit.
- ☐ 4. Keep environment-specific configuration out of the artifact.
- ☐ 5. Intentionally break compilation, one unit test, and package restoration. Record where and how each failure surfaces.
- ☐ 6. Stretch: run the same pipeline on a self-hosted agent and inventory everything the agent requires.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

ADR-001: Build Artifacts Are Immutable. Use Context, Decision, Alternatives, and Consequences.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

What on your self-hosted agent is implicit rather than declared?

Could another engineer recreate the build machine from your documentation?

What would make an artifact impossible to reproduce later?

Knowledge check

1. Why should a build artifact be immutable?

2. Why is building independently in every environment weaker than artifact promotion?

3. What should uniquely identify an artifact?

4. What is the difference between an agent and a pipeline?

5. Which build dependencies should be explicitly declared?

6. What risks arise when self-hosted agents accumulate undocumented software?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 3

Continuous Deployment and Artifact Promotion

Practice promoting one artifact through environments and separating deployment success from environment readiness.

Suggested time	Primary output	Status
6-7 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Promote the same artifact through DEV and QA.
- ☐ Externalize environment configuration.
- ☐ Capture deployment provenance.
- ☐ Use post-deployment health validation and approvals.

Resources

AZ-400: Secure continuous deployment (Selected modules) - Covers deployment patterns, release gates, configuration, identity, and secure pipeline design.

<https://learn.microsoft.com/en-us/training/paths/az-400-implement-secure-continuous-deployment/>

Mission-critical deployment and testing (Reference) - Use for continuous validation and reliability-oriented deployment thinking.

<https://learn.microsoft.com/en-us/azure/well-architected/mission-critical/mission-critical-deployment-testing>

Hands-on lab

- ☐ 1. Extend the pipeline to Build -> Artifact -> Deploy DEV -> DEV health check -> Approval -> Deploy QA -> QA health check.
- ☐ 2. Build only once.
- ☐ 3. Supply environment-specific configuration during deployment.
- ☐ 4. Record artifact ID, commit, branch, build time, deployment time, target environment, and validation result.
- ☐ 5. Deliberately deploy a wrong connection string, missing database, invalid configuration, and unavailable dependency.

[] 6. For each case, record separately whether deployment execution succeeded and whether the environment was ready.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

A diagram titled "Deployment Success vs. Environment Readiness" plus a small deployment metadata schema.

Completion checklist

- [] I completed the required reading/tutorial selections.
- [] The lab runs end-to-end and I can explain the result.
- [] The required deliverable is committed to the repository.
- [] I can explain at least one tradeoff I made.
- [] I completed the knowledge check without consulting the answer key.
- [] I wrote a brief weekly retrospective.

Reflection prompts

What conditions should block promotion?

Which validation checks belong immediately after deployment?

Could your validation run even if a different deployment tool were used?

Knowledge check

1. Can a deployment technically succeed while the environment is unusable? Explain.

2. What is artifact promotion?

3. What information establishes deployment provenance?

4. Should environment validation be part of the artifact build? Why or why not?

5. Why might post-deployment validation be independent of the deployment tool?

6. What is a deployment gate?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 4

Architecture Communication and ADRs

Learn to explain a system at the right level and make technical decisions durable and reviewable.

Suggested time	Primary output	Status
5-6 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Use C4-style abstraction levels appropriately.
- ☐ Create context, container, and deployment views.
- ☐ Write concise ADRs that expose tradeoffs.
- ☐ Separate architectural decisions from implementation detail.

Resources

The C4 Model (60-90 min) - Official introduction to hierarchical software architecture diagrams.
<https://c4model.com/>

Architecture Decision Record repository (45-60 min) - ADR explanations, templates, examples, and conventions.
<https://github.com/architecture-decision-record/architecture-decision-record>

Azure Architecture Center - Architecture guide (Reference) - Reference for quality attributes and architectural guidance.
<https://learn.microsoft.com/en-us/azure/architecture/guide/>

Hands-on lab

- ☐ 1. Create a system context diagram for Developer, QA, Policy System, Azure DevOps, Database, and Monitoring.
- ☐ 2. Create a container diagram for Web API, SQL Database, Pipeline, Validation Service, and Telemetry Platform.
- ☐ 3. Create a deployment diagram showing DEV and QA as separate environments.
- ☐ 4. Write ADR-002: Independent Post-Deployment Validation.
- ☐ 5. Write ADR-003: Deployment Metadata Storage and compare JSON file, database, pipeline variable, and central deployment registry.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

A mini architecture packet: 3 diagrams plus ADR-002 and ADR-003.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which diagram would you show an executive? Which would you show an engineer implementing the system?

What decision is expensive enough to deserve an ADR?

Which implementation details are intentionally absent from your diagrams?

Knowledge check

1. When should you create an ADR?

2. Why should an architecture diagram avoid showing every class?

3. What question does a system-context diagram answer?

4. What is a quality attribute?

5. Why is maintainability an architecture concern?

6. What is the difference between architecture and implementation?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 5

Observability with OpenTelemetry

Instrument a system so that failures can be understood from evidence rather than server-by-server investigation.

Suggested time	Primary output	Status
6-7 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Distinguish logs, metrics, traces, health checks, and alerts.
- ☐ Instrument ASP.NET Core with OpenTelemetry concepts.
- ☐ Correlate telemetry with deployment identity.
- ☐ Use telemetry to answer concrete operational questions.

Resources

OpenTelemetry .NET (Reference) - Official .NET language documentation.

<https://opentelemetry.io/docs/languages/dotnet/>

OpenTelemetry .NET Getting Started (60-90 min) - Hands-on ASP.NET Core instrumentation tutorial.

<https://opentelemetry.io/docs/languages/dotnet/getting-started/>

.NET observability with OpenTelemetry (45 min) - Microsoft guidance for .NET observability APIs and OpenTelemetry.

<https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel>

Hands-on lab

- ☐ 1. Add structured logs for policy issuance requested, policy issuance succeeded, and database failure.
- ☐ 2. Add metrics for total policy issues, failures, and duration.
- ☐ 3. Add traces covering HTTP request -> validation -> database query -> policy creation.
- ☐ 4. Attach deployment version and environment to telemetry.
- ☐ 5. Break the database and answer: Which request failed? Why? Which component failed? Which deployment was running? How long did the request take?
- ☐ 6. Improve instrumentation until all five questions can be answered from telemetry.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

An Observability Contract defining required deployment attributes, request correlation, critical workflow metrics, and error context.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which telemetry signal gave you the fastest answer?

What information would you regret not capturing during an incident?

How would you prevent high-cardinality telemetry from becoming noisy or expensive?

Knowledge check

1. What is a log?

2. What is a metric?

3. What is a trace?

4. How is an event different from an alert?

5. What is a health check?

6. Why is "we have logs" not equivalent to "the system is observable"?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 6

Reliability Engineering and Failure Modes

Shift from asking whether the happy path works to deliberately reasoning about how systems fail and recover.

Suggested time	Primary output	Status
5-6 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Use failure-mode analysis to reason systematically.
- ☐ Differentiate availability, resilience, and recovery.
- ☐ Design detection and recovery alongside normal functionality.
- ☐ Practice failure injection in a controlled environment.

Resources

Azure Well-Architected Framework - Reliability (1 hr 29 min) - Structured module covering business requirements, resilience, recovery, operations, and simplicity.

<https://learn.microsoft.com/en-us/training/modules/azure-well-architected-reliability/>

Reliability quick links (Reference) - Reference hub for reliability principles, failure-mode analysis, testing, and targets.

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/>

Reliability design patterns (Reference) - Use as a pattern catalog after completing your own failure analysis.

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/design-patterns>

Hands-on lab

- ☐ 1. Create an FMEA table with Component, Failure, Effect, Detection, and Recovery.
- ☐ 2. Include at least: API process down, database unavailable, schema mismatch, invalid configuration, and dependency timeout.
- ☐ 3. For each failure, predict behavior before injecting it.
- ☐ 4. Inject the failure, observe actual behavior, compare it with your prediction, and improve detection.
- ☐ 5. Add one failure that affects only part of the user workflow and design graceful degradation if possible.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

Failure Mode and Effects Analysis plus a one-page "Reliability Improvements" backlog ranked by risk.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which failure surprised you most?

Which failure had the longest detection delay?

Which recovery depends on undocumented human knowledge?

Knowledge check

1. What is resilience?

2. How is resilience different from availability?

3. Why is recovery part of architecture?

4. What is graceful degradation?

5. Why should teams intentionally test failure conditions?

6. Why is simplicity a reliability technique?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 7

Site Reliability Engineering: SLI, SLO, Error Budgets, and Toil

Turn reliability from a vague aspiration into measurable service behavior and prioritized engineering work.

Suggested time	Primary output	Status
5-6 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Define meaningful SLIs and SLOs.
- ☐ Explain why 100% reliability is usually the wrong target.
- ☐ Use error budgets conceptually to balance change and reliability.
- ☐ Identify and prioritize operational toil.

Resources

Google Site Reliability Engineering (Selected chapters) - Read selected chapters on monitoring, SLOs, automation, and incident learning.

<https://sre.google/sre-book/table-of-contents/>

Google SRE Workbook (Selected chapters) - Use the SLO and monitoring chapters as practical guidance.

<https://sre.google/workbook/table-of-contents/>

Hands-on lab

- ☐ 1. Define three SLIs for your sample service.
- ☐ 2. For each SLI, record Measurement, Target, Window, Failure Criteria, and Data Source.
- ☐ 3. Include at least one user-facing workflow SLI such as successful policy issuance.
- ☐ 4. Define one deployment-oriented SLI such as percentage of QA deployments that pass health validation.
- ☐ 5. Identify 10 repetitive operational tasks and score Frequency, Time, Human Judgment Required, Automation Potential, and Risk.
- ☐ 6. Rank the toil candidates and choose one for automation.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

One-page Reliability Scorecard plus a ranked Toil Reduction Backlog.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which metric best represents the user experience rather than infrastructure health?

What behavior would a 100% SLO accidentally discourage?

Which toil item is frequent but should not be automated because it requires judgment?

Knowledge check

1. What is an SLI?

2. What is an SLO?

3. What is an SLA?

4. Why should an SLO usually be below 100%?

5. What is an error budget?

6. What characteristics make operational work toil?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 8

Environment Certification Service

Combine delivery, observability, and reliability concepts into a deployment-independent validation capability.

Suggested time	Primary output	Status
7-8 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Design an independent environment certification model.
- ☐ Create health checks spanning application, data, configuration, and synthetic workflows.
- ☐ Emit machine-readable certification results.
- ☐ Keep validation loosely coupled to deployment execution.

Resources

Azure Well-Architected - Operational Excellence (1 hr 22 min) - Focus on observability, automation, standards, supply chain, and safe deployments.

<https://learn.microsoft.com/en-us/training/modules/azure-well-architected-operational-excellence/>

OpenTelemetry .NET (Reference) - Reuse the observability model from Week 5.

<https://opentelemetry.io/docs/languages/dotnet/>

Hands-on lab

- ☐ 1. Build an Environment Certification Service that accepts environment and deployment identity as input.
- ☐ 2. Implement checks for application reachable, database reachable, expected schema, expected configuration, expected deployment version, critical dependency, and synthetic policy transaction.
- ☐ 3. Return a machine-readable PASS/FAIL result per check plus overall status.
- ☐ 4. Persist certification history.
- ☐ 5. Trigger certification manually and after deployment.
- ☐ 6. Prove that the same certification can run after an Azure DevOps deployment, a PowerShell deployment, or a simulated manual deployment.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

Working certification prototype plus ADR-004: Validation Is Independent of Deployment Execution.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

What is the minimum evidence required to call an environment READY?

Which checks are destructive or state-changing?

How would certification behave if one non-critical dependency is degraded?

Knowledge check

1. If a deployment system reports SUCCESS and certification reports FAIL, can both be correct? Explain.

2. Why is validation independence valuable during a deployment-platform migration?

3. What makes a synthetic transaction different from a simple health endpoint?

4. What information should a certification record retain?

5. What should happen if the certification system itself is unavailable?

6. Which checks should be blocking versus advisory?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 9

Architecture Proposal and Decision Enablement

Practice compressing technical reasoning into a form that helps leaders choose among tradeoffs.

Suggested time	Primary output	Status
5-6 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Write a concise engineering proposal.
- ☐ Separate evidence, constraints, options, recommendation, and tradeoffs.
- ☐ Move implementation detail into appendices.
- ☐ Summarize a proposal verbally in 60 seconds.

Resources

Azure Well-Architected - Operational Excellence (Review selected units) - Review standardization, automation, observability, and safe deployment practices as decision criteria.

<https://learn.microsoft.com/en-us/training/modules/azure-well-architected-operational-excellence/>

Architecture Decision Records (Reference) - Use ADR thinking to make tradeoffs explicit.

<https://github.com/architecture-decision-record/architecture-decision-record>

Hands-on lab

- ☐ 1. Pretend leadership asks: "How should we improve deployment reliability?"
- ☐ 2. Write a two-page maximum proposal using Problem, Evidence, Constraints, Options, Recommendation, Tradeoffs, Rollout, and Success Criteria.
- ☐ 3. Compare at least: do nothing, add independent validation, standardize current deployment, and replace deployment platform.
- ☐ 4. Cut the proposal from two pages to one page without losing the decision structure.
- ☐ 5. Create a 60-second verbal summary with one explicit decision request.
- ☐ 6. Move technical details that do not affect the decision into an appendix.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

Two-page proposal, one-page executive version, and 60-second decision pitch.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Did your recommendation make the tradeoff obvious?

Which paragraph could be removed without weakening the decision?

What question would a skeptical executive ask first?

Knowledge check

1. What is the difference between explaining a technical solution and enabling a decision?

2. What belongs in the main recommendation versus an appendix?

3. Why should alternatives be presented even when you strongly prefer one option?

4. What makes a decision request actionable?

5. Why are constraints important in a technical proposal?

6. How can success criteria protect an initiative from becoming open-ended?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

WEEK 10

Technical Leadership Simulation

Integrate the previous nine weeks into an architecture review that demonstrates technical judgment, communication, and ownership.

Suggested time	Primary output	Status
6-8 hours	Portfolio artifact	☐ Not started ☐ In progress ☐ Complete

Learning outcomes

- ☐ Present architecture and tradeoffs to mixed technical/non-technical stakeholders.
- ☐ Answer skeptical questions concisely.
- ☐ Connect implementation decisions to organizational capability.
- ☐ Produce a portfolio-quality capstone package.

Resources

Azure Well-Architected Framework learning path (Reference/review) - Use as a final architecture quality checklist.

<https://learn.microsoft.com/en-us/training/paths/azure-well-architected-framework/>

C4 Model (Reference) - Review abstraction and diagram audience.

<https://c4model.com/>

Google SRE Workbook (Reference) - Review reliability and operational sections relevant to your design.

<https://sre.google/workbook/table-of-contents/>

Hands-on lab

- ☐ 1. Prepare a 15-minute architecture review: 2 min Problem, 2 min Current Architecture, 3 min Proposed Architecture, 3 min Decisions/Tradeoffs, 2 min Reliability Model, 2 min Migration Plan, 1 min Decision Requested.
- ☐ 2. Ask another person or ChatGPT to challenge you with skeptical questions.
- ☐ 3. Answer each challenge in under 60 seconds.
- ☐ 4. Revise the architecture or proposal when a challenge reveals a real weakness rather than merely defending the original design.
- ☐ 5. Record your final review if useful and critique clarity, pace, jargon, and decision framing.

Required deliverable

SUBMIT TO YOUR PORTFOLIO

Capstone architecture review deck/packet and completed final assessment described below.

Completion checklist

- ☐ I completed the required reading/tutorial selections.
- ☐ The lab runs end-to-end and I can explain the result.
- ☐ The required deliverable is committed to the repository.
- ☐ I can explain at least one tradeoff I made.
- ☐ I completed the knowledge check without consulting the answer key.
- ☐ I wrote a brief weekly retrospective.

Reflection prompts

Which question exposed the weakest part of your design?

Where did you use too much implementation detail?

What did you change because of feedback?

What evidence now demonstrates a repeatable technical-leadership capability?

Knowledge check

1. Why is "who owns this?" an architecture question as well as an organizational question?

2. How would you justify an independent validation layer if the deployment pipeline already has tests?

3. What is the risk of making a near-term reliability control dependent on a multi-year platform migration?

4. How should you respond when a stakeholder proposes solving a detection problem only with training/process?

5. What does a good migration plan preserve while changing architecture?

6. What is the difference between defending your design and exercising technical judgment?

Weekly retrospective

What became clearer this week?

What was harder than expected?

What would I do differently in a production system?

What artifact from this week best demonstrates growth?

Final Capstone Assessment

SCENARIO

A company has 30 independently customized customer applications. Deployment methods vary between teams. QA frequently discovers broken environments after deployment. Some configuration changes occur outside source control. Leadership wants to standardize deployment eventually, but migration will take 18 months.

Design a solution and produce the following evidence:

- ☐ System context diagram
- ☐ Deployment architecture
- ☐ Validation architecture
- ☐ CI/CD model
- ☐ Observability model
- ☐ Reliability metrics (SLIs/SLOs)
- ☐ Ownership model
- ☐ Rollout strategy
- ☐ Three architecture decision records
- ☐ Two-page executive proposal
- ☐ 15-minute architecture review with an explicit decision request

Scoring rubric

Dimension	Developing	Strong	Excellent
Problem framing	Describes symptoms	Separates causes, constraints, and outcomes	Frames the decision and identifies residual risk
Architecture	Shows components	Shows boundaries, flows, and deployment model	Connects architecture to quality attributes and migration
Reliability	Lists checks	Defines failure modes and recovery	Defines measurable targets and validates assumptions
Tradeoffs	Presents one solution	Compares credible options	Explains why the recommendation fits

			current constraints
Communication	Technically correct	Clear to mixed audiences	Concise, decision-oriented, and adaptable under challenge
Leadership	Completes assigned work	Defines next steps and ownership	Creates a reusable capability others can adopt

Knowledge Check Answer Key

Use these answers as comparison points, not scripts. A correct answer in your own words is better evidence of understanding than reproducing this wording.

Week 1: Think in Delivery Systems

1. What is the difference between continuous integration and continuous delivery?

CI continuously integrates and verifies source changes; CD extends the process toward packaging, deployment, and/or release in a repeatable way.

2. Why is a deployment pipeline more than a build script?

A deployment pipeline coordinates artifact movement, environment configuration, controls, verification, and often approvals - not just compilation.

3. Why can human intervention increase deployment risk?

Manual steps are inconsistent, hard to audit, and can be skipped or performed differently across runs.

4. What is the difference between preventing a failure and detecting one?

Prevention tries to stop an invalid state from occurring; detection identifies that an invalid state exists.

5. What information would you need to identify exactly what code is running in QA?

At minimum: artifact/build ID, source commit, branch or source reference, deployment timestamp, and target environment.

6. Why can deployment validation be treated separately from deployment execution?

Because a deployment mechanism can finish successfully while the resulting environment is still unhealthy or misconfigured.

Week 2: Continuous Integration and Reproducible Builds

1. Why should a build artifact be immutable?

Immutability ensures the exact bytes that were tested are the bytes promoted later.

2. Why is building independently in every environment weaker than artifact promotion?

Separate builds can produce different outputs because source, dependencies, tooling, or configuration may differ.

3. What should uniquely identify an artifact?

A stable artifact/build ID tied to source provenance such as commit SHA and build metadata.

4. What is the difference between an agent and a pipeline?

A pipeline defines work; an agent is the compute environment that executes jobs from that definition.

5. Which build dependencies should be explicitly declared?

Compilers/runtimes, SDKs, package sources, package versions, test adapters, build tools, and scripts.

6. What risks arise when self-hosted agents accumulate undocumented software?

Builds become non-reproducible, upgrades become risky, hidden dependencies accumulate, and failures are harder to diagnose.

Week 3: Continuous Deployment and Artifact Promotion

1. Can a deployment technically succeed while the environment is unusable? Explain.

Yes. Files may copy and services may start even though configuration, schema, dependencies, or workflows are unhealthy.

2. What is artifact promotion?

Artifact promotion moves the same previously built artifact through successive environments rather than rebuilding it.

3. What information establishes deployment provenance?

Artifact ID, source commit, source branch/ref, deployment timestamp, target environment, and ideally deployment actor/pipeline/run.

4. Should environment validation be part of the artifact build? Why or why not?

Usually no. Build-time verification proves properties of the artifact; environment validation proves properties of the deployed system and its dependencies.

5. Why might post-deployment validation be independent of the deployment tool?

Loose coupling lets the same validation protect multiple deployment mechanisms and survive platform migration.

6. What is a deployment gate?

A rule or automated/manual check that must pass before a deployment or promotion can continue.

Week 4: Architecture Communication and ADRs

1. When should you create an ADR?

When a significant technical choice has lasting consequences, alternatives, or tradeoffs that future engineers need to understand.

2. Why should an architecture diagram avoid showing every class?

Too much detail obscures system boundaries, responsibilities, relationships, and decisions.

3. What question does a system-context diagram answer?

What system are we discussing, who uses it, and what external systems does it interact with?

4. What is a quality attribute?

A measurable or evaluable characteristic such as reliability, security, performance, maintainability, or operability.

5. Why is maintainability an architecture concern?

Maintainability shapes boundaries, coupling, change cost, testing strategy, ownership, and long-term system evolution.

6. What is the difference between architecture and implementation?

Architecture defines important structures, boundaries, constraints, and decisions; implementation realizes those choices in concrete code and configuration.

Week 5: Observability with OpenTelemetry

1. What is a log?

A timestamped record describing something that happened, ideally with structured context.

2. What is a metric?

A numeric measurement aggregated over time, useful for trends, rates, thresholds, and objectives.

3. What is a trace?

A correlated representation of work flowing through components, composed of spans and timing/context relationships.

4. How is an event different from an alert?

An event is something that occurred; an alert is a notification generated because observed conditions crossed a meaningful rule or threshold.

5. What is a health check?

A targeted probe that reports whether a system or dependency meets a defined readiness/liveness condition.

6. Why is "we have logs" not equivalent to "the system is observable"?

Observability requires enough signals and context to infer internal state and answer new questions, not merely a collection of text records.

Week 6: Reliability Engineering and Failure Modes

1. What is resilience?

Resilience is the ability to withstand, adapt to, and recover from failures while preserving acceptable service.

2. How is resilience different from availability?

Availability is the proportion of time/service accessible; resilience describes behavior before, during, and after faults.

3. Why is recovery part of architecture?

Recovery requirements affect data, redundancy, deployment, state management, backups, ownership, and operational design.

4. What is graceful degradation?

Continuing to provide a reduced but useful capability when part of the system is impaired.

5. Why should teams intentionally test failure conditions?

Failure testing validates assumptions, exposes hidden dependencies, measures detection/recovery, and builds operational confidence.

6. Why is simplicity a reliability technique?

Every dependency, state transition, and special case increases possible failure modes and cognitive load.

Week 7: Site Reliability Engineering: SLI, SLO, Error Budgets, and Toil

1. What is an SLI?

An SLI is a quantitative measurement of service behavior such as availability, latency, correctness, or freshness.

2. What is an SLO?

An SLO is a target for an SLI over a defined window.

3. What is an SLA?

An SLA is a formal commitment, often contractual, with consequences if the target is missed.

4. Why should an SLO usually be below 100%?

Perfect targets are often unrealistic, expensive, and can discourage useful change while providing no room for tradeoffs.

5. What is an error budget?

The reliability tolerance implied by the SLO - the amount of failure/change risk the service can absorb before the objective is missed.

6. What characteristics make operational work toil?

It is repetitive, manual, automatable, tactical, scales with service growth, and provides limited enduring value compared with engineering work.

Week 8: Environment Certification Service

1. If a deployment system reports SUCCESS and certification reports FAIL, can both be correct? Explain.

Yes. The deployment execution may have completed exactly as designed while the resulting system fails readiness criteria.

2. Why is validation independence valuable during a deployment-platform migration?

The validation capability can protect current and future deployment mechanisms without being rewritten for each one.

3. What makes a synthetic transaction different from a simple health endpoint?

A synthetic transaction exercises a real user/business path and multiple dependencies rather than only testing process liveness.

4. What information should a certification record retain?

Deployment identity, environment, timestamp, each check/result, durations, relevant evidence, and overall status.

5. What should happen if the certification system itself is unavailable?

The release policy should define fail-open or fail-closed behavior based on risk; the certification system itself should also be monitored and recoverable.

6. Which checks should be blocking versus advisory?

Blocking checks should represent conditions that make testing/use unsafe or invalid; advisory checks indicate degradation or risk that can be consciously accepted.

Week 9: Architecture Proposal and Decision Enablement

1. What is the difference between explaining a technical solution and enabling a decision?

Explanation transfers understanding; decision enablement organizes enough evidence, options, tradeoffs, and consequences for someone to choose and act.

2. What belongs in the main recommendation versus an appendix?

Decision-relevant evidence, constraints, options, recommendation, tradeoffs, rollout, and success criteria belong in the main body; deep implementation details belong in appendices.

3. Why should alternatives be presented even when you strongly prefer one option?

Alternatives demonstrate that tradeoffs were considered and clarify why the recommendation is preferable under current constraints.

4. What makes a decision request actionable?

It identifies the specific choice, owner/authority, scope, and immediate next action needed.

5. Why are constraints important in a technical proposal?

Constraints prevent unrealistic options and explain why a locally imperfect solution may be strategically correct.

6. How can success criteria protect an initiative from becoming open-ended?

They define measurable completion/learning conditions, bound scope, and create a point at which the initiative can be evaluated.

Week 10: Technical Leadership Simulation

1. Why is "who owns this?" an architecture question as well as an organizational question?

Ownership determines decision rights, operations, escalation, change control, support boundaries, and therefore system behavior over time.

2. How would you justify an independent validation layer if the deployment pipeline already has tests?

Pipeline tests may verify artifact or deployment execution; independent validation verifies the actual resulting environment and can protect multiple deployment mechanisms.

3. What is the risk of making a near-term reliability control dependent on a multi-year platform migration?

The organization can remain exposed to known failures while waiting for a much larger dependency to complete, and the smaller initiative can stall indefinitely.

4. How should you respond when a stakeholder proposes solving a detection problem only with training/process?

Acknowledge prevention value, then separate prevention from detection and explain which residual failure modes remain even with perfect compliance.

5. What does a good migration plan preserve while changing architecture?

Service continuity, traceability, rollback/recovery options, measurable acceptance criteria, and a path for incremental learning.

6. What is the difference between defending your design and exercising technical judgment?

Defending assumes the design must win; judgment updates the design when better evidence, constraints, or tradeoffs emerge.

Resource Index

Explore Azure Pipelines - Short orientation to Azure Pipelines terminology and core concepts.

<https://learn.microsoft.com/en-us/training/modules/explore-azure-pipelines/>

DevOps foundations - Review DevOps as a system of people, process, automation, feedback, and continuous delivery.

<https://learn.microsoft.com/en-us/training/paths/devops-foundations-core-principles-practices/>

Azure DevOps overview - Use as a reference for CI/CD and delivery-system concepts.

<https://learn.microsoft.com/en-us/devops/overview/>

AZ-400: Implement CI with Azure Pipelines and GitHub Actions - Primary learning path for pipeline configuration and agent management.

<https://learn.microsoft.com/en-us/training/paths/az-400-implement-ci-azure-pipelines-github-actions/>

Azure Pipelines agents - Pay particular attention to hosted vs. self-hosted agent responsibilities.

<https://learn.microsoft.com/en-us/azure/devops/pipelines/agents/agents>

AZ-400: Secure continuous deployment - Covers deployment patterns, release gates, configuration, identity, and secure pipeline design.

<https://learn.microsoft.com/en-us/training/paths/az-400-implement-secure-continuous-deployment/>

Mission-critical deployment and testing - Use for continuous validation and reliability-oriented deployment thinking.

<https://learn.microsoft.com/en-us/azure/well-architected/mission-critical/mission-critical-deployment-testing>

The C4 Model - Official introduction to hierarchical software architecture diagrams.

<https://c4model.com/>

Architecture Decision Record repository - ADR explanations, templates, examples, and conventions.

<https://github.com/architecture-decision-record/architecture-decision-record>

Azure Architecture Center - Architecture guide - Reference for quality attributes and architectural guidance.

<https://learn.microsoft.com/en-us/azure/architecture/guide/>

OpenTelemetry .NET - Official .NET language documentation.

<https://opentelemetry.io/docs/languages/dotnet/>

OpenTelemetry .NET Getting Started - Hands-on ASP.NET Core instrumentation tutorial.

<https://opentelemetry.io/docs/languages/dotnet/getting-started/>

.NET observability with OpenTelemetry - Microsoft guidance for .NET observability APIs and OpenTelemetry.

<https://learn.microsoft.com/en-us/dotnet/core/diagnostics/observability-with-otel>

Azure Well-Architected Framework - Reliability - Structured module covering business requirements, resilience, recovery, operations, and simplicity.

<https://learn.microsoft.com/en-us/training/modules/azure-well-architected-reliability/>

Reliability quick links - Reference hub for reliability principles, failure-mode analysis, testing, and targets.

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/>

Reliability design patterns - Use as a pattern catalog after completing your own failure analysis.

<https://learn.microsoft.com/en-us/azure/well-architected/reliability/design-patterns>

Google Site Reliability Engineering - Read selected chapters on monitoring, SLOs, automation, and incident learning.

<https://sre.google/sre-book/table-of-contents/>

Google SRE Workbook - Use the SLO and monitoring chapters as practical guidance.

<https://sre.google/workbook/table-of-contents/>

Azure Well-Architected - Operational Excellence - Focus on observability, automation, standards, supply chain, and safe deployments.

<https://learn.microsoft.com/en-us/training/modules/azure-well-architected-operational-excellence/>

Azure Well-Architected Framework learning path - Use as a final architecture quality checklist.

<https://learn.microsoft.com/en-us/training/paths/azure-well-architected-framework/>

Suggested Personal Review Cadence

[] End of Week 3: Review whether you can explain artifact promotion and environment readiness without notes.

[] End of Week 6: Review whether you naturally ask "how will this fail?" during design.

[] End of Week 8: Review whether your certification design is deployment-tool independent.

[] End of Week 10: Compare your first-week delivery diagram with your final architecture. Write one page on how your thinking changed.

Program Completion

DONE WHEN

You can take an ambiguous delivery/reliability problem, structure it into architecture and operational concerns, build a working slice, explain its failure modes and observability, present alternatives, and make a concise recommendation with measurable success criteria.